

By Solving Problems

1. Early Stopping
2. Data Scaling
3. Dropout layer
4. Regularization

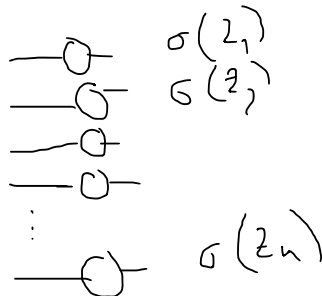
By fine tuning the hyper parameters.

1. No. of epochs
2. No. of HL
3. No. of Nodes / HL
4. Learning rate
5. Batch size
6. Activation function
7. Optimizer

Fine tuning:-

1. Activation Func^r

→ Zero centered Sigmoid



$$\text{mean}(\sigma(z_1) + \sigma(z_2) + \dots + \sigma(z_n)) \neq 0$$

$$w_n = w_0 - \eta \frac{dL}{dw}$$

$$w_n = w_0 - \eta \Delta w$$

$$\Delta w = \frac{w_0 - w_n}{\eta}$$

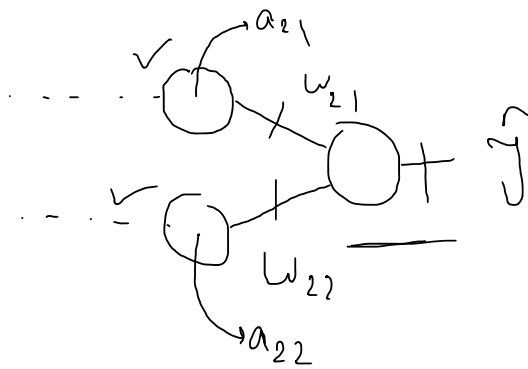
2 3 4 4 5

$$\text{mean} = \frac{2+3+4+4+5}{5} = 3.6$$

$$\text{mean} = \frac{2+3+4+4+5}{5} = 3.6$$

$$\text{mode} = 4(2)$$

$$\text{median} = 4$$



$$\frac{dL}{dw_{21}} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dz_{31}} \cdot \frac{dz_{31}}{dw_{21}}$$

$$\boxed{\frac{dL}{dw_{21}} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dz_{31}} \cdot \frac{a_{21}}{w_{21} + a_{22}w_{11} + b_{31}}}$$

$$z_{31} = a_{21}w_{21} + a_{22}w_{11} + b_{31}$$

normalized gradient

mean=0

$$\frac{dz_{31}}{dw_{21}} = a_{21}$$

$$\frac{dL}{dw_{21}} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dz_{31}} \cdot \frac{a_{21}}{w_{21} + a_{22}w_{11} + b_{31}}$$

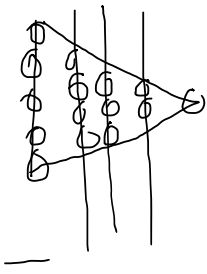
→ tanh (-1, 1)

→ zero centered

2. ^{non} Saturating → Relu

1. f HL and Nodes Per HL

2. Number of HL and Nodes for HL



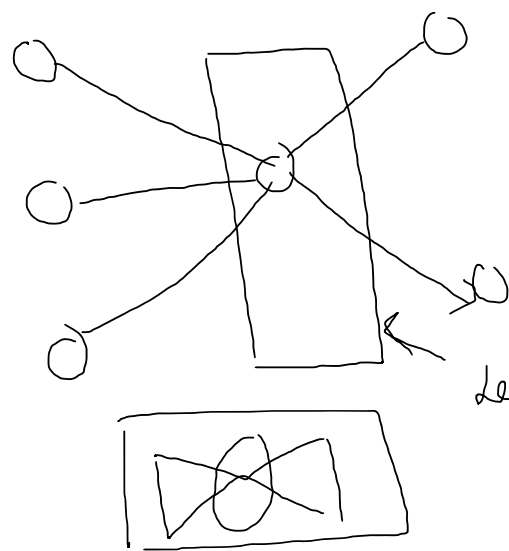
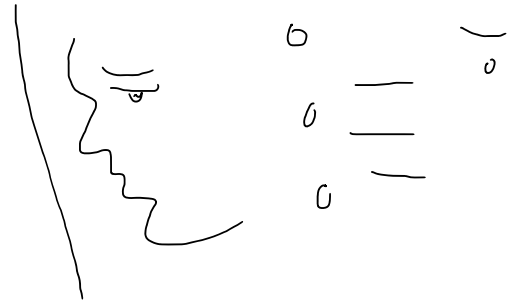
Machine Learning → Feature Engineering

DL → FE ✓

Primitive Feature
Extract
Storing layer

→ Shape

Later layer



① More than Sufficient nodes and HL

② Priority HL count.

less Feature Extracted

Feature Loss

↪ cannot be recovered

3. choosing Rate and Batch Size

3. Learning



Stochastic GD

time ↑



GPU RAM X

Mini Batch Batch Size

- large (20-25) → fast accuracy ↑
- Small (4-8) → time ↑ slow accuracy ↑

Epochs → Early Stopping (Stable Training)

Large Batch Size:

Learning Rate Scheduling

starting epochs → lr ↓

later on " → lr ↑↑

fast ✓
accuracy ✓

Ⓜ Problems?

1. Vanishing / Exploding
2. Not enough data
3. Slow training

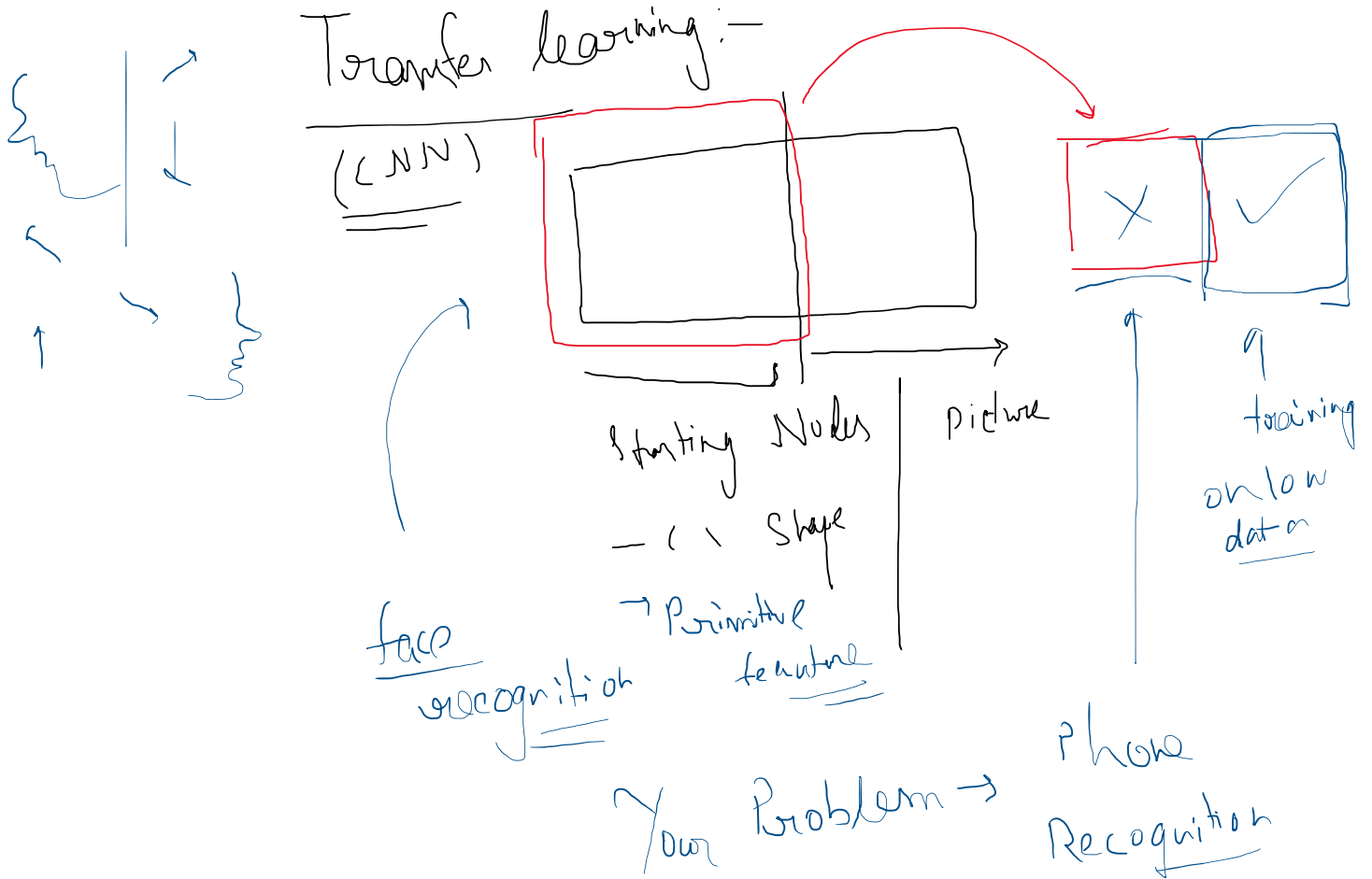
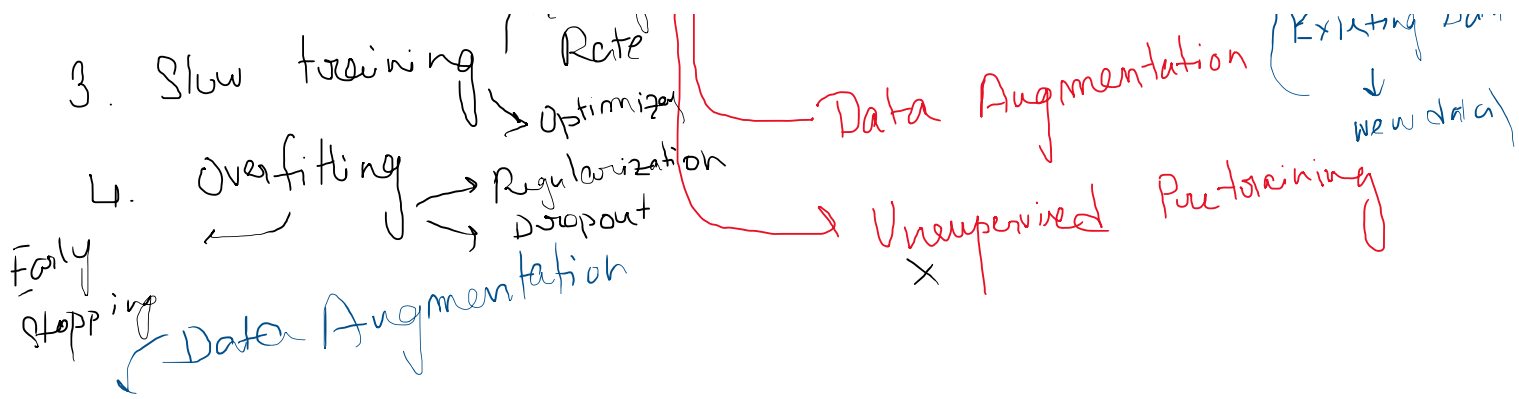
gradient

Transfer learning

ReLU

Weight Init techniques
Activation Func
Batch Normalization
gradient clipping

Augmentation (Existing Data ↓)



Early Stopping

20 December 2025 21:54

Early stopping is a crucial deep learning technique to prevent **overfitting** by monitoring a model's performance on a separate **validation set** and halting training when validation error starts rising (or accuracy drops), even as training error decreases

Overfitting

Training ✓
Test ✗

Args	
monitor	Quantity to be monitored. Defaults to "val_loss".
min_delta	Minimum change in the monitored quantity to qualify as an improvement, i.e. an absolute change of less than min_delta, will count as no improvement. Defaults to 0.
patience	Number of epochs with no improvement after which training will be stopped. Defaults to 0.
verbose	Verbosity mode, 0 or 1. Mode 0 is silent, and mode 1 displays messages when the callback takes an action. Defaults to 0.
mode	One of {"auto", "min", "max"}. In min mode, training will stop when the quantity monitored has stopped decreasing; in "max" mode it will stop when the quantity monitored has stopped increasing; in "auto" mode, the direction is automatically inferred from the name of the monitored quantity. Defaults to "auto".
baseline	Baseline value for the monitored quantity. If not None, training will stop if the model doesn't show improvement over the baseline. Defaults to None.
restore_best_weights	Whether to restore model weights from the epoch with the best value of the monitored quantity. If False, the model weights obtained at the last step of training are used. An epoch will be restored regardless of the performance relative to the baseline. If no epoch improves on baseline, training will run for patience epochs and restore weights from the best epoch in that set. Defaults to False.
start_from_epoch	Number of epochs to wait before starting to monitor improvement. This allows for a warm-up period in which no improvement is expected and thus training will not be stopped. Defaults to 0.

Example:

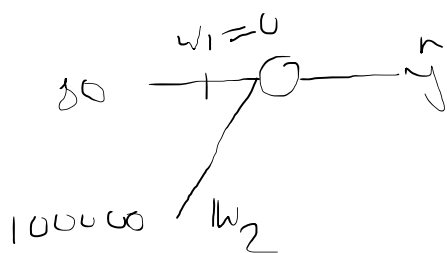
```
>>> callback = keras.callbacks.EarlyStopping(monitor='loss',
...                                           patience=3)
>>> # This callback will stop the training when there is no improvement in
>>> # the loss for three consecutive epochs.
>>> model = keras.models.Sequential([keras.layers.Dense(10)])
>>> model.compile(keras.optimizers.SGD(), loss='mse')
>>> history = model.fit(np.arange(100).reshape(5, 20), np.zeros(5),
...                    epochs=10, batch_size=1, callbacks=[callback],
...                    verbose=0)
>>> len(history.history['loss']) # Only 4 epochs are run.
4
```

x (0-400)	income (0-100)	loan (0-100)	$\hat{y}$ (Y/Y)
80	1L	20L	50L
120	50L	100	
$\downarrow$	$\downarrow$		
range	range		

Problem if i/p's are not normalized!

Weights?

Input column jump outance



$$L = (\hat{y} - y) \quad \text{MAE}$$

$$L = (50 - 20) = 30$$

$$\hat{y} = 80w_1 + 100000w_2$$

$$\frac{dL}{dw_1} = \left[\frac{dL}{d\hat{y}} \right] \frac{d\hat{y}}{dw_1} = 30 \times 80 = \boxed{2400}$$

$$\frac{dL}{dw_2} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dw_2} = 30 \times 100000 = \boxed{3000000}$$

$$\eta = 0.01$$

$$w_1 = w_1 - \eta (2400) = 0 - 24 = -24$$

$$\boxed{w_1 = -24}$$

$$w_2 = w_2 - \eta \quad 36L = 0 - 36k = \boxed{-36000}$$

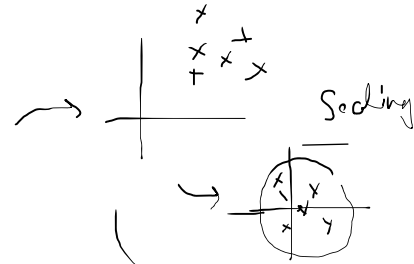
$$\boxed{w_2 = -36k}$$

$$\boxed{w_1 = w_2 = 0}$$

$$\boxed{w_2 \gg w_1}$$

(Salary) $\rightarrow \mu = 0$
 $\rightarrow$ Standardization $\rightarrow \sigma = 1$

$$\frac{X_i - \mu}{\sigma}$$

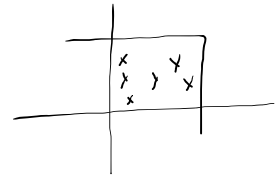


Scaling

$\rightarrow$ Normalization
 (Age)

$$\frac{X_i - X_{\min}}{X_{\max} - X_{\min}}$$

mean



After Normalization

$\rightarrow (0, 1)$

age	income	loan	
0.01	0.9	20	50
0.4	0.02		

relation
 $w_1 \quad w_2$

$$0.01 \quad w_1 = 0$$

$$0.030 \quad w_2 = 0$$

$$L = (50 - 20) = 30$$

$$\hat{y} = 0.01 w_1 + 0.03 w_2 + b_1$$

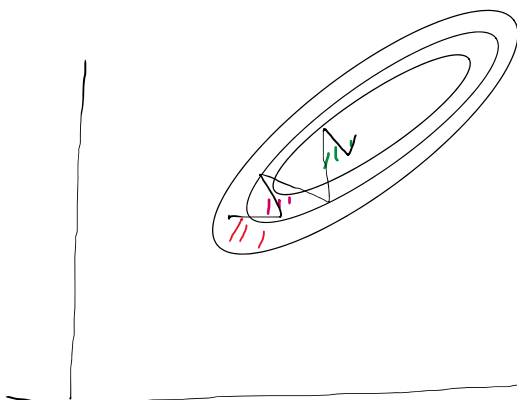
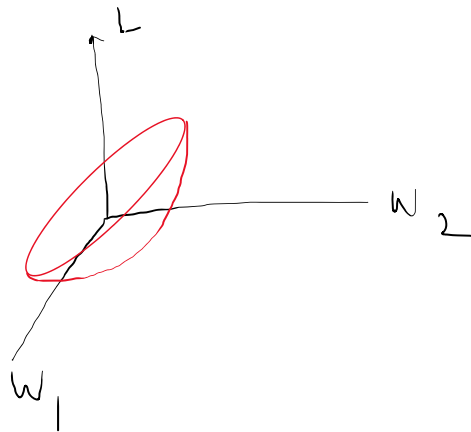
$$\frac{dL}{dw_1} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dw_1} = 30 \cdot (0.001) = \boxed{0.03}$$

$$\frac{dL}{dw_2} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dw_2} = 30 \times (0.9) = 2.7$$

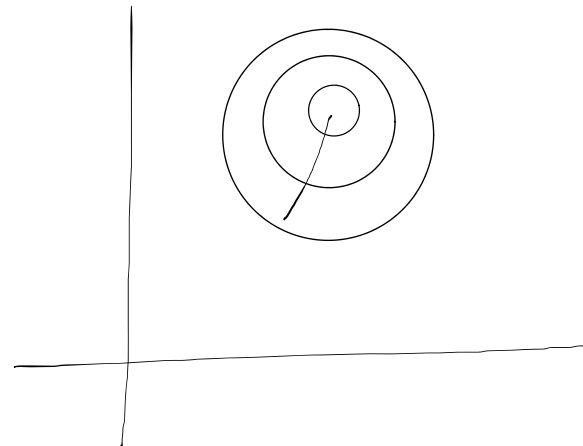
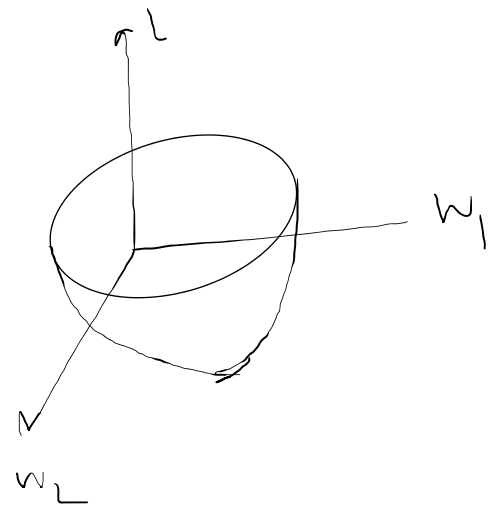
$$w_1 = 0 - 0.01 \times 0.03 = -0.0003$$

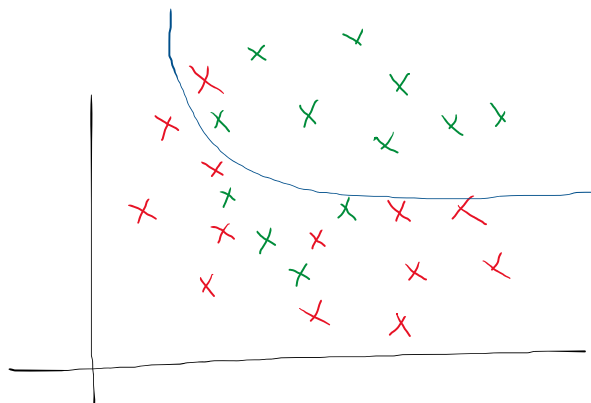
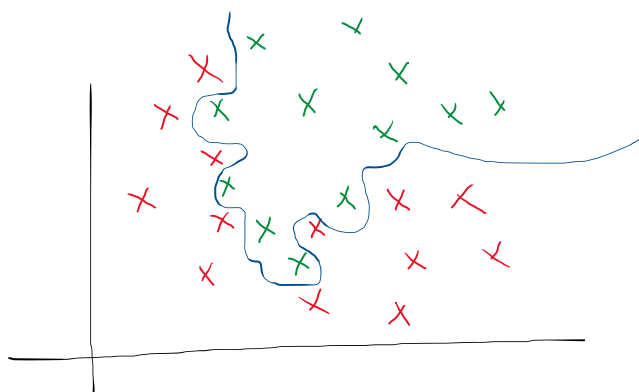
$$w_2 = 0 - 0.01 \times 2.7 = -0.027$$

Without Data Scaling

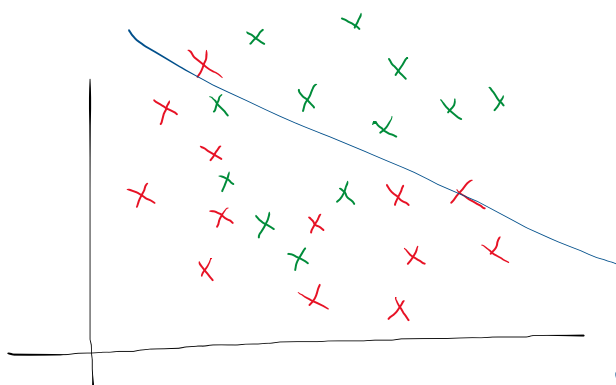


With Data Scaling

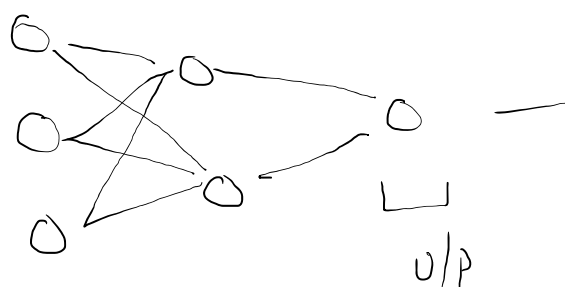
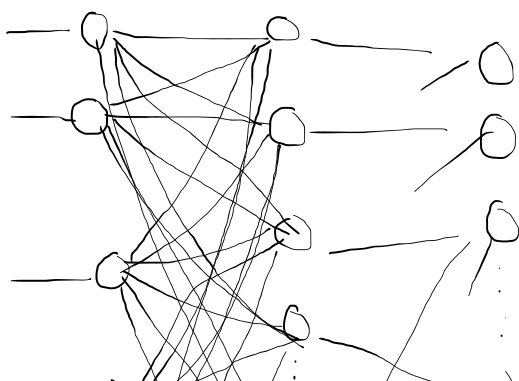


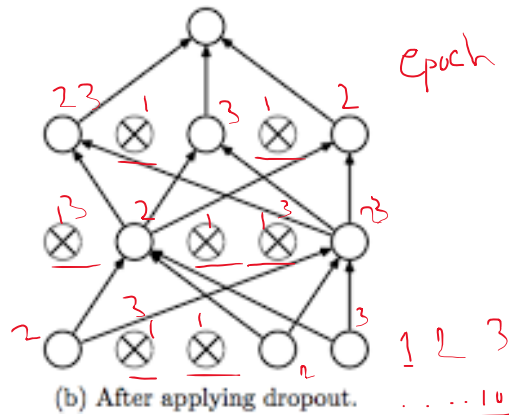
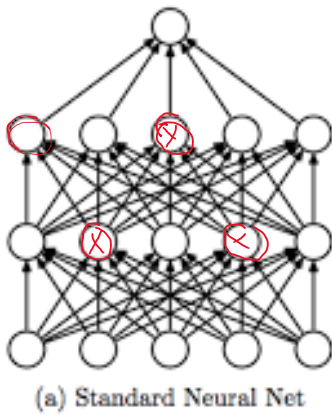
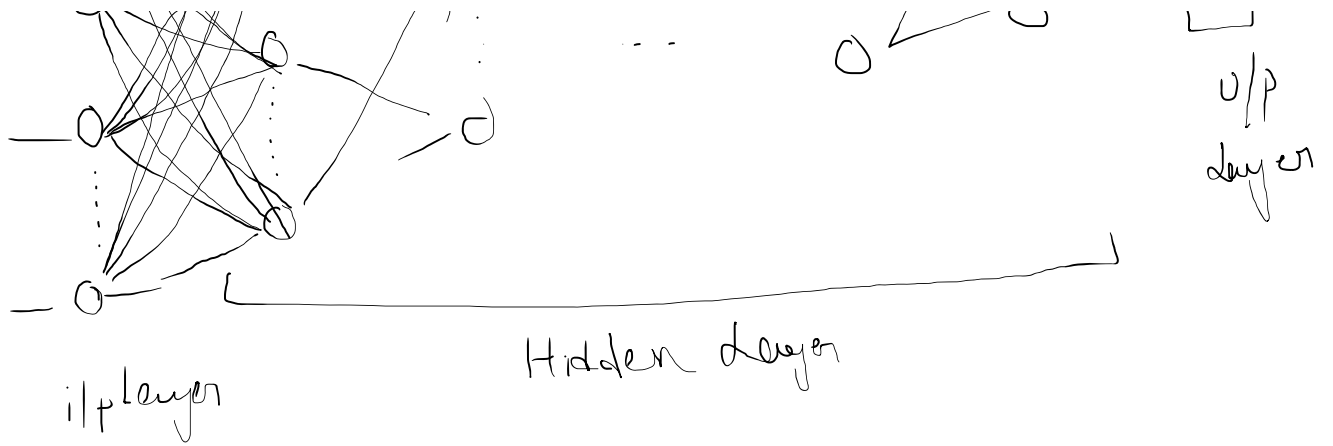


[Overfitting] $\uparrow$ nodes = power to capture more nonlinearity. Normal



Underfitting $\xrightarrow{\text{Sol}^n}$ give more data





epoch = (1 → 10)

i) i/p
ii) Hidden layer
Turn off some nodes randomly.

UNIQUE
10 10 10

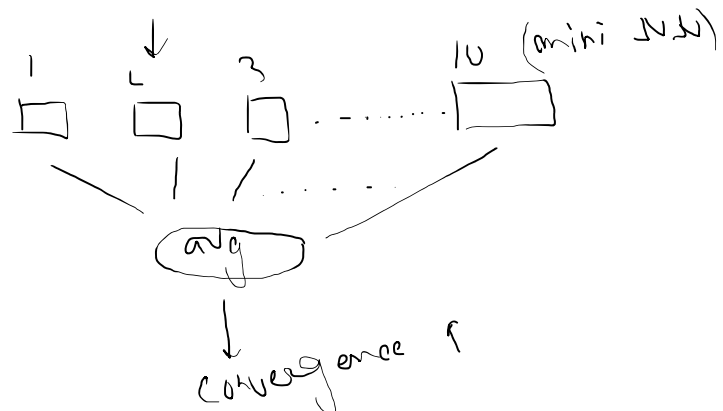
1. Prevents overfitting
Neuron dependency ↓
↑ generalization of the data

3. Controlled Randomness into training

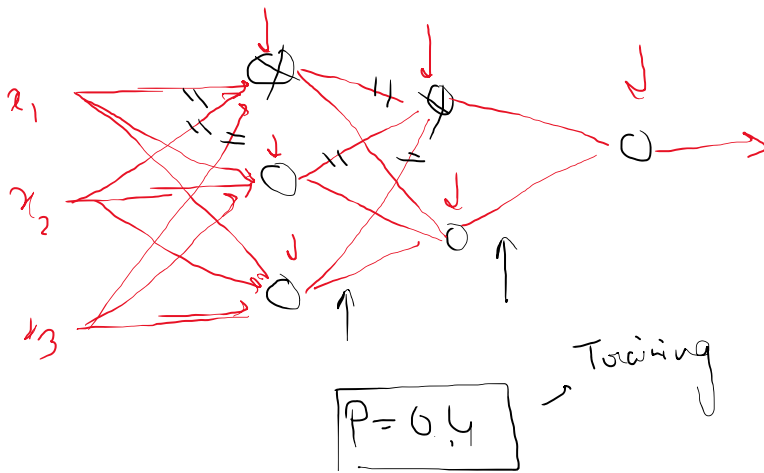
Controlled noise

↳ (sensitivity towards small change ↓↓)

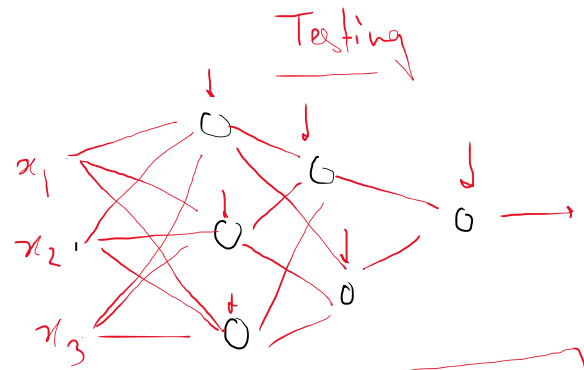
2. Ensemble Effect
(Wisdom of crowd)
(KBC, D adagrad)



* Dropout only works during training



$$W$$



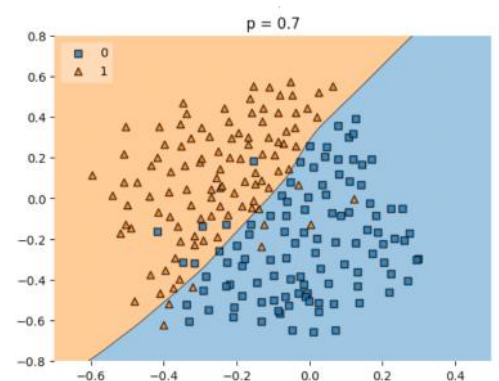
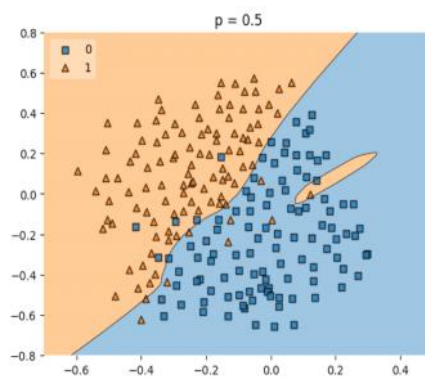
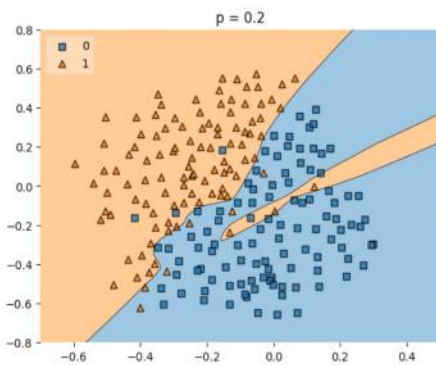
(w) Training Testing!

$$W_{\text{test}} = W_{\text{train}} (1-p)$$

$$W_{\text{test}} = W_{\text{training}} \times (1-p)$$

$$W_{\text{Test}} = W_{\text{train}} \times 0.6$$

(Prediction)



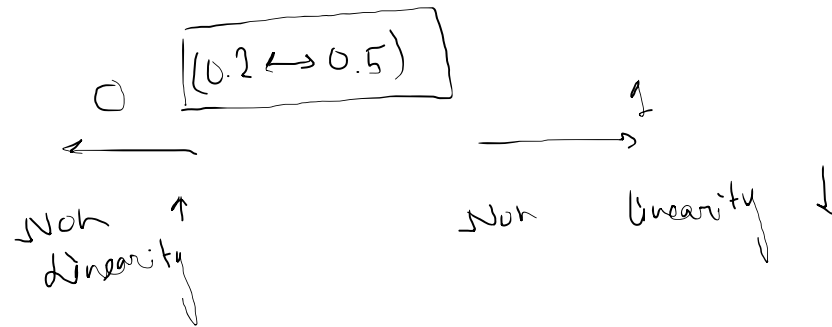
$$w = w(1 - 0.2)$$

$$w = 0.8 * w$$

$$w = 0.5 * w$$

$$[0.2, 0.5]$$

$$w = 0.3 * w$$



Weights $\rightarrow \uparrow$
 small i/p change (sensitive)

$$\hat{y} \approx 2x$$

Training

x	y
1	2.1
2	3.9
3	6.2

Testing

x	y
4	8.0

$$\hat{y} = wx$$

Single Param.

(MSE)

$$L = \sum (y - wx)^2$$

$$\frac{1}{2} \sum (y - \hat{y})^2 = L$$

$$w = 2.04$$

Testing

$$\hat{y} = 2.04 \times 4 = 8.16$$

$$L = \sum_{i=1}^n (y_i - wx_i)^2 = \sum_{i=1}^n y_i^2 - 2w \sum_{i=1}^n x_i y_i + w^2 \sum_{i=1}^n x_i^2$$

$$\frac{dL}{dw} = 0 - 2 \sum_{i=1}^n x_i y_i + 2w \sum_{i=1}^n x_i^2 = 0$$



$$\frac{dL}{dw} = 0 - 2 \sum_{i=1}^n x_i y_i + 2w \sum_{i=1}^n x_i^2 = 0$$

$$w = \frac{\sum x_i y_i}{\sum x_i^2} = \frac{X^T Y}{X^T X}$$

$$w = \frac{X^T Y}{X^T X}$$

$$X^T Y = 1(2.1) + 2(3.5) + 3(6.4)$$

$$= 28.5$$

$$X^T X = 1^2 + 2^2 + 3^2 = 14$$

$$w = \frac{28.5}{14} \approx 2.04$$

$$[x] = \begin{bmatrix} y \\ \vdots \end{bmatrix}$$

Slightly Noisy data

x

y

1

3.5

2

1.2

3

7.8

$$w = \frac{X^T Y}{X^T X}$$

$$= \frac{1 \times (3.5) + 2(1.2) + 3(7.8)}{1^2 + 2^2 + 3^2}$$

$$= \frac{29.3}{14} \approx 2.1$$

$$\hat{y} =$$

$$2.1 \times 4 = 8.4$$

For Deep NN $W=5$

$$\boxed{\hat{y} = 20}$$

Regularization

$$L_{\text{new}} = L_{\text{old}} + \text{Penalty term}$$

$w \downarrow$

→ L2 Reg (DL ↑)

→ L1 Reg

L2 Regularization

$$C = \frac{1}{n} \sum_{i=1}^m L(y_i, \hat{y}_i) + \text{Penalty term}$$

hyperparameter → $\frac{\lambda}{2n} \sum_{i=1}^m w_i^2$

Sum of Square of weights

for easy calculation

$$C = \frac{1}{n} L(y, \hat{y}) + \sum_{k=1}^K \sum_{i=1}^I \sum_{j=1}^J \frac{\|w_{ij}^k\|^2}{n}$$

made don

$$C = \frac{1}{n} L(y, g_i) + \sum_{l=1}^L \sum_{i=1}^{n_l} \sum_{j=1}^{n_{l+1}} w_{ij}$$

$\uparrow$ no. of layers $\uparrow$ prev node pos $\uparrow$ cur node pos

Solution $w_0' = w_0 - \eta \frac{dL'}{dw_0} \quad \left| \quad L' = L + \frac{\lambda}{2} \sum \|w_i\|^2 \right.$

$$\frac{dL'}{dw_0} = \frac{dL}{dw_0} + \frac{\lambda}{2} \cdot 2 \cdot w_0$$

$$w_0' = w_0 - \eta \left(\frac{dL}{dw_0} + \lambda w_0 \right)$$

$$= \underline{w_0} - \eta \frac{dL}{dw_0} - \eta \lambda w_0$$

$$\boxed{\frac{dL'}{dw_0} = \frac{dL}{dw_0} + \lambda w_0}$$

$$\boxed{w_0' = (1 - \eta \lambda) w_0 - \eta \frac{dL}{dw_0}}$$

$\uparrow$ with Regularization (L_2)

$$\boxed{w_0}$$

$$\boxed{\eta \lambda \neq 1}$$

Ques 1
 $\boxed{10 \times 10 \times 10}$

$$\boxed{w_n = w_0 - \eta \frac{dL}{dw_0}} \quad \text{without}$$

for L_2 Regularization

$$= \frac{1}{n} L(y, \hat{y}) + \lambda \sum_{i=1}^n w_i$$

for -

$$C = \frac{1}{n} \sum L(y, \hat{y}) + \lambda \sum_{i=1}^n w_i$$

$$w'_0 = w_0 - \eta \frac{dL'}{dw} \quad \left| \quad L' = L + \lambda \sum_{i=1}^n w_i \right.$$

$$= w_0 - \eta \left(\frac{dL}{dw_0} + \lambda \right) \quad \left| \quad \frac{dL'}{dw_0} = \frac{dL}{dw_0} + \lambda \right.$$

w_0